Cidades ESG Inteligentes

Implementação DevOps

Projeto: EnergyGuard API

Monitoramento de eficiência energética e pegada de carbono

Integrantes do grupo

Eduardo Henrique Leite Rodrigues **RM564950**

Eduardo de Melo Albanese **RM561790**

Disciplina: Navegando pelo mundo DevOps

Data: 13/10/2026

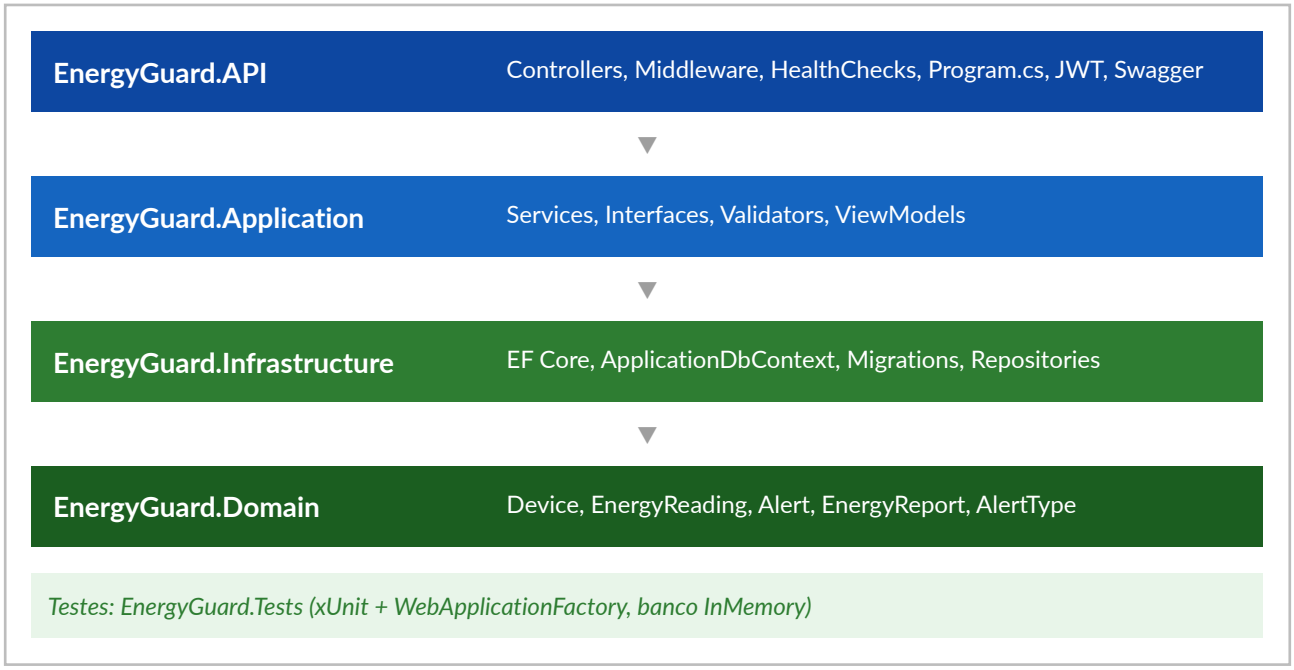
1. Introdução e objetivo

Este documento descreve a implementação DevOps da API EnergyGuard, uma aplicação ASP.NET Core 7 de monitoramento de eficiência energética. O objetivo é entregar uma esteira completa de integração e entrega contínuas (CI/CD), containerização, orquestração e documentação, preservando integralmente a aplicação original e sua finalidade ESG.

O tema ESG é atendido pelo pilar Ambiental (E): a aplicação mede consumo de energia (kWh), sinaliza desperdícios por meio de alertas e consolida relatórios com a pegada de carbono (CarbonFootprintKg), apoiando metas de eficiência e descarbonização em cidades inteligentes.

2. Arquitetura da solução

A solução segue arquitetura em camadas (Clean Architecture), com separação clara de responsabilidades entre domínio, aplicação, infraestrutura e API:



Correção estrutural: no projeto recebido, o projeto de Infraestrutura estava dentro da pasta de Testes, enquanto todos os .csproj o referenciavam como projeto irmão — a solução não compilava. A infraestrutura foi reposicionada no nível correto e foi criada a solução EnergyGuard.sln. Nenhuma regra de negócio, endpoint ou entidade foi alterada.

3. Tecnologias utilizadas

- .NET 7 / ASP.NET Core 7 (Web API REST)
- Entity Framework Core 7 + SQL Server (persistência e migrations)
- ASP.NET Identity + JWT (autenticação e autorização)
- FluentValidation (validação de entrada) e Swagger (documentação da API)

- xUnit + WebApplicationFactory (testes de integração)
- Docker e Docker Compose (containerização e orquestração)
- GitHub Actions + GitHub Container Registry / GHCR (CI/CD e registry)

4. Explicação do pipeline CI/CD

O pipeline (arquivo .github/workflows/ci-cd.yml) é acionado por push e pull request nas branches main, master e develop, além de execução manual. Utiliza permissões mínimas (contents: read, packages: write). Fluxo:



- build-test: restore, build em Release e execução dos testes (relatório .trx + cobertura). Falha de build ou teste interrompe o pipeline.
- container-smoke-test: na própria nuvem do GitHub, sobe API + banco (docker compose), valida / health/ready e demonstra staging e produção — gera evidência real sem Docker local.
- docker-build-push: login no GHCR e publicação da imagem com tags imutáveis por SHA.
- deploy-staging e deploy-production: promoção via SSH + docker compose em servidor real (opcional), com verificação de readiness. Produção exige aprovação manual.

5. Etapas de build, testes e deploy

Build e testes são executados localmente e no CI. O deploy reutiliza a mesma imagem versionada por SHA em staging e produção, garantindo consistência entre ambientes.

```
dotnet restore EnergyGuard.sln
dotnet build EnergyGuard.sln -c Release --no-restore
dotnet test EnergyGuard.sln -c Release --collect:"XPlat Code Coverage"
```

6. Arquitetura Docker e Docker Compose

O Dockerfile usa build multi-stage (imagens oficiais aspnet:7.0 e sdk:7.0), executa a aplicação como usuário não-root (appuser, UID 1001), expõe a porta 8080 e define um HEALTHCHECK via curl em / health/live. O docker-compose.yml orquestra dois serviços: api e db (SQL Server 2022).

- Serviço api: construído a partir do Dockerfile, depende do db saudável.
- Serviço db: SQL Server com healthcheck via sqlcmd; não exposto publicamente.

7. Redes, volumes e variáveis de ambiente

- Rede dedicada energyguard-net (bridge): a API acessa o banco pelo hostname interno 'db'.
- Volume mssql-data (mapeado em /var/opt/mssql): persiste os dados após reinicializações.

- Variáveis via arquivo .env (fora do versionamento); .env.example documenta todas elas.
- Migrations aplicadas no startup (Database.Migrate() com retry); ignoradas no provedor InMemory dos testes.

8. Estratégia de separação dos ambientes

Os ambientes são definidos por arquivos compose complementares que reutilizam a mesma imagem validada (variável IMAGE, preferencialmente a tag por SHA do GHCR):

- docker-compose.staging.yml: ASPNETCORE_ENVIRONMENT=Staging, porta host 8081, volume próprio.
- docker-compose.production.yml: ASPNETCORE_ENVIRONMENT=Production, porta host 8082, restart: always, volume próprio.

Rollback: por usar tags imutáveis por SHA, basta redefinir IMAGE para o SHA anterior e repetir docker compose pull && up -d.

9. Comandos utilizados

```
# Build + testes (sem Docker)
dotnet test EnergyGuard.sln -c Release

# Subir localmente (com Docker)
docker compose up -d --build
docker compose ps
curl http://localhost:8080/health/ready
docker compose down
```

```
# Windows (PowerShell)
.\scripts\test.ps1
.\scripts\start.ps1 -Environment base -Build
.\scripts\health.ps1 -Environment base
.\scripts\validate-environments.ps1   # demo staging(8081)+prod(8082)
```

10. Evidências reais de execução

As evidências reais obtidas no ambiente de preparação estão em docs/evidencias/logs/:

Evidência	Status	Local / Obtenção
Build da aplicação	REAL	logs/01-build.log (0 erros)
Testes automatizados	REAL	logs/02-tests.log (12/12)
Sintaxe YAML (workflow+compose)	REAL	logs/03-yaml-validation.log
Ambiente .NET	REAL	logs/00-dotnet-info.log
Build da imagem Docker	PENDENTE	GitHub Actions (push) ou docker build
Contêineres + saúde	PENDENTE	Actions: job container-smoke-test

Staging e produção	PENDENTE	Actions (smoke-test) / servidor real

Build: 0 erros, 0 avisos. Testes: 12 aprovados / 12 (0 falhas). Os endpoints de saúde são exercitados de verdade pelos testes de integração (HealthCheckTests).

11. Desafios encontrados e soluções aplicadas

- Solução não compilava (Infraestrutura aninhada em Testes): projeto reposicionado como irmão e criação de EnergyGuard.sln.
- Ausência de endpoint de saúde: implementados /health/live (liveness), /health/ready (readiness com verificação de banco) e /health (agregado).
- .NET 7 escuta na porta 80 por padrão: fixado ASPNETCORE_URLS=http://+:8080 no Dockerfile.
- Inicialização do banco: Database.Migrate() com retry para tolerar o tempo de subida do contêiner de SQL Server.
- Instabilidade momentânea de DNS no NuGet durante o restore: resolvida com nova tentativa.

12. Limitações e pré-requisitos para execução

- O ambiente de preparação NÃO possui Docker: build da imagem, execução de contêineres e deploy não foram executados aqui (apenas configurados e validados por inspeção).
- Build e testes .NET foram executados de verdade (evidências reais anexadas).
- Deploy remoto depende de servidores com Docker e dos secrets/variáveis do GitHub (DEPLOY_ENABLED, *_SSH_*, *_URL); sem eles, os jobs de deploy são ignorados por design.
- .NET 7 está fora de suporte; foi preservado conforme o projeto original (recomenda-se migração futura para .NET 8 LTS).

13. Checklist de entrega

- [x] Projeto compactado em ZIP com estrutura organizada — *Entrega_DevOps_Cidades_ESG.zip*
- [x] Dockerfile funcional (multi-stage, não-root, healthcheck) — *autoria validada; build pendente de Docker*
- [x] Docker Compose (base + staging + produção) — *autoria validada; execução pendente de Docker*
- [x] Pipeline com build, testes e deploy — *build/testes prontos; deploy gated por infra/secrets*
- [x] README com instruções e evidências
- [x] Documentação técnica em PDF — *este documento*
- [ ] Deploy em staging e produção COMPROVADO — *PENDENTE — requer Docker/servidores*

Documento gerado automaticamente a partir da arquitetura efetivamente implementada. Nenhum print, log, URL ou resultado de execução foi fabricado.